

BCC202 - Estruturas de Dados I

Aula 24: Tabelas *Hash*

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Abordagens de pesquisa em Memória Primária

- ▶ Pesquisa Sequencial.
- ▶ Pesquisa Binária.
- ▶ Árvores de Pesquisa:
 - ▶ Árvores Binárias de Pesquisa.
 - ▶ Árvores AVL.
- ▶ Transformação de Chave (*Hashing*):
 - ▶ *Hashing* Perfeito.
 - ▶ Listas Encadeadas.
 - ▶ Endereçamento Aberto.

Conteúdo

Introdução

Hashing Perfeito

Hashing Imperfeito

Funções de Transformação comuns

Listas Encadeadas

Considerações Finais

Bibliografia

Introdução

Introdução

- ▶ A base dos métodos de pesquisa vistos até o momento são baseados em **comparação** de chaves.
 - ▶ Os algoritmos tinham por base a **ordenação** dos dados.
 - ▶ Utilizam da ordenação dos dados para realizar uma pesquisa **eficiente**.
 - ▶ Esforço computacional de $O(\log n)$.
- ▶ A solução é *hashing* (tabela de dispersão) ou método de cálculo de endereço.
 - ▶ No caso médio é possível encontrar a chave em tempo **constante**.

Tabela *Hash*

Hash significa:

- ▶ Fazer picadinho de carne e vegetais para cozinhar.
- ▶ Fazer uma bagunça. (**Webster's New World Dictionary**)
- ▶ Espalhar x Transformar (informática x computação).

Tabela Hash

Hashing significa:

- ▶ Utiliza-se índices em arranjos ou listas para acessar alguma informação.
- ▶ O acesso é feito nas posições do array em tempo $O(1)$.
- ▶ Mas para acessar/buscar uma posição não é em tempo $O(1)$.
- ▶ Uma solução são as Tabelas Hashs (estrutura de dados Hash).

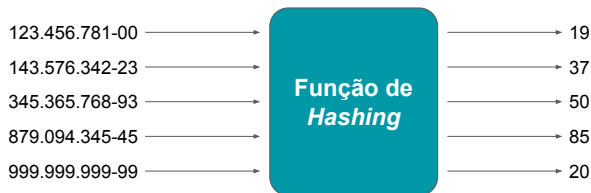
Em uma Tabela Hash, os registros armazenados em um vetor são diretamente endereçados a partir de uma **transformação aritmética sobre a chave de pesquisa**.

Tabela Hash

Mas o que é?

- ▶ Uma estrutura de dados especial.
- ▶ Armazena as informações desejadas associando **chaves**.
- ▶ O objetivo é, a partir de uma **chave**, fazer uma busca rápida para obter o valor desejado.

Exemplo de Hash

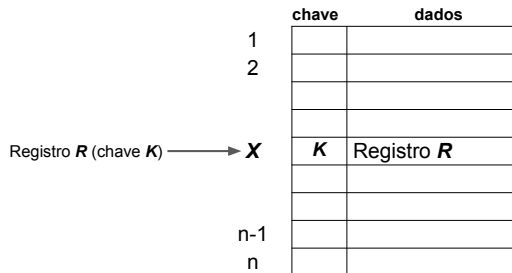


19	123.456.781-00; Igor Hassum; Av. Ribeiro. N° 13.
20	
...	
37	143.576.342-23; Carla Soares; Rua Marechal Oliveira. N° 27.
...	
50	345.365.768-93; Silvio Santos; Av. SBT. S/N
...	
85	879.094345-45; Rubens Barrichello; Rua B. N° 100.
...	

Hashing e dicionários

- ▶ Dicionário é uma estrutura **especializada** para otimizar operações de inserir, pesquisar e remover.
- ▶ *Hash* acaba sendo uma “**generalização**” mais simples do tipo **dicionário**.
- ▶ Utiliza-se uma **função de transformação de chave** (*hashing*) para retornar o **índice** onde deve (deveria) estar armazenada a informação.

Ilustração de uma Tabela *Hash*



- ▶ Registro *R* tem uma chave *K*.
- ▶ Armazena-se na posição *X*.
- ▶ Mas como foi determinado essa posição *X*?

Ilustração de uma Tabela *Hash*

	chave	dados
1		
2		
Registro <i>R</i> (chave <i>K</i>) → X	<i>K</i>	Registro <i>R</i>
n-1		
n		

- ▶ Registro *R* tem uma chave *K*.
- ▶ Armazena-se na posição *X*.
- ▶ Mas como foi determinado essa posição *X*?
 - ▶ **Função de Hashing.**

Como representar Tabelas *Hash*?

Vetor

- ▶ Cada posição armazena uma informação.
- ▶ *Hashing* Perfeito.
- ▶ Endereçamento Aberto.

Vetor + Lista Encadeada

- ▶ Cada posição do vetor contém um ponteiro para as listas que armazenam as informações.

Transformação de Chave (*Hashing*)

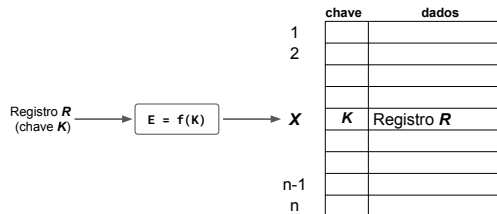
Um método de pesquisa com o uso da transformação de chave é constituído de duas etapas principais:

1. Computar o valor da **função de transformação**, a qual transforma a chave de pesquisa em um endereço da tabela.
2. Considerando que duas ou mais chaves podem ser transformadas em um mesmo endereço de tabela, é necessário existir um método para lidar com **colisões**.

Funções de Transformação (*Hashing*)

- ▶ Uma função de transformação h deve **mapear** chaves em inteiros dentro do intervalo $[0, \dots, M - 1]$, onde M é o tamanho da tabela.
- ▶ A função de transformação ideal é aquela que:
 - ▶ Seja **simples** de ser computada.
 - ▶ Para cada chave de entrada, qualquer uma das saídas possíveis é **igualmente provável de ocorrer**.
- ▶ **Idealmente** é que essa função forneça **índices únicos** para o conjunto das chaves de entrada possíveis.
- ▶ A função de *Hashing* é **extremamente importante**, pois ela é responsável por distribuir as informações pela Tabela *Hash*.

Ilustração da Função *Hashing*



- ▶ Os valores da chave podem ser:
 - ▶ **Numéricos.**
 - ▶ **Alfabéticos.**
 - ▶ **Alfa-numéricos.**

- ▶ A transformação se dá pela transformação do valor de uma chave em um endereço.
- ▶ Podem ser aplicadas operações aritméticas e/ou lógicas.

$$f : C \rightarrow E$$

f: Função de cálculo de endereço.

C: Espaço de valores da chave (domínio de f).

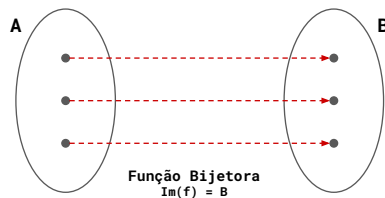
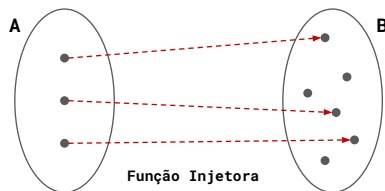
E: Espaço de endereçamento (contradomínio de f).

Hashing Perfeito

Hashing Perfeito

Considerando:

- ▶ Duas chaves diferentes x_1 e x_2 .
- ▶ Ambas pertencentes a um conjunto A .
- ▶ A função utilizada sempre fornece resultados **diferentes**.



Exemplo de *Hashing* Perfeito

- ▶ Cada aluno de uma turma específica será armazenado em uma Tabela *Hash*.
- ▶ A identificação de um aluno se dá por sua matrícula.

```
1 | typedef struct {  
2 |     int mat;  
3 |     .....  
4 | } Aluno;
```

- ▶ A matrícula tem 7 números (xx.x.xxxx).
- ▶ Queremos acessar qualquer aluno em **ordem constante**. Como fazer?

Exemplo de *Hashing* Perfeito

- ▶ Cada aluno de uma turma específica será armazenado em uma Tabela *Hash*.
- ▶ A identificação de um aluno se dá por sua matrícula.

```
1 | typedef struct {  
2 |     int mat;  
3 |     .....  
4 | } Aluno;
```

- ▶ A matrícula tem 7 números (xx.x.xxxx).
- ▶ Queremos acessar qualquer aluno em **ordem constante**. Como fazer?
 - ▶ Usar a própria **matrícula para acessar** o vetor.

Exemplo de Hashing Perfeito

- ▶ Cada aluno de uma turma específica será armazenado em uma Tabela *Hash*.
- ▶ A identificação de um aluno se dá por sua matrícula.

```
1 | typedef struct {  
2 |     int mat;  
3 |     .....  
4 | } Aluno;
```

- ▶ A matrícula tem 7 números (xx.x.xxxx).
- ▶ Queremos acessar qualquer aluno em **ordem constante**. Como fazer?
 - ▶ Usar a própria **matrícula para acessar** o vetor.
- ▶ Há um problema grande para ter esse acesso com custo contante. Qual?

Exemplo de *Hashing* Perfeito

- ▶ Cada aluno de uma turma específica será armazenado em uma Tabela *Hash*.
- ▶ A identificação de um aluno se dá por sua matrícula.

```
1 | typedef struct {  
2 |     int mat;  
3 |     .....  
4 | } Aluno;
```

- ▶ A matrícula tem 7 números (xx.x.xxxx).
- ▶ Queremos acessar qualquer aluno em **ordem constante**. Como fazer?
 - ▶ Usar a própria **matrícula para acessar** o vetor.
- ▶ Há um problema grande para ter esse acesso com custo contante. Qual?
 - ▶ 7 dígitos = matrículas entre 0.000.000 a 9.999.999.
 - ▶ Dez milhões de elementos.

Problema de Memória de *Hashing* Perfeito

```
1 typedef struct {  
2     int mat;           // 4 bytes = 4 bytes  
3     char nome[81];     // 1 byte * 81 = 81 bytes  
4     char email[41];    // 1 byte * 41 = 41 bytes  
5 } Aluno;              // Total = 126 bytes
```

- ▶ Cada estrutura de cada aluno ocupa 126 bytes.
 - ▶ $10.000.000 \times 126 \text{ bytes} = 1.260.000.000 \text{ bytes} \approx 1 \text{ GByte}$ de memória.
- ▶ Normalmente são somente 50/60 alunos em uma turma.
 - ▶ $50 * 126 \text{ bytes} = 7.300 \text{ bytes} \approx 7 \text{ Mbyte}$ de memória.
 - ▶ **Desperdício** de memória.
- ▶ Pergunta: Como **economizar/diminuir** desperdício de alocação?

Resolvendo o desperdício de espaço

- ▶ Utilizar um vetor de **ponteiros** ao invés de um vetor de estruturas.
- ▶ Os alunos serão cadastrados somente dinamicamente.
 - ▶ Cada posição teria um único aluno cadastrado.
- ▶ Como cada ponteiro ocupa 4 bytes o gasto será de ????

Resolvendo o desperdício de espaço

- ▶ Utilizar um vetor de **ponteiros** ao invés de um vetor de estruturas.
- ▶ Os alunos serão cadastrados somente dinamicamente.
 - ▶ Cada posição teria um único aluno cadastrado.
- ▶ Como cada ponteiro ocupa 4 bytes o gasto será de **40.000.000 bytes** ($\approx 38\text{Mbs}$).
- ▶ Há outra estratégia?

Resolvendo o desperdício de espaço

- ▶ Utilizar um vetor de **ponteiros** ao invés de um vetor de estruturas.
- ▶ Os alunos serão cadastrados somente dinamicamente.
 - ▶ Cada posição teria um único aluno cadastrado.
- ▶ Como cada ponteiro ocupa 4 bytes o gasto será de **40.000.000 bytes** ($\approx 38\text{Mbs}$).
- ▶ Há outra estratégia? SIM!
- ▶ Pegar somente as **partes significativas** da chave.

12.3.4567

Análise sobre o *Hashing* perfeito

- Vale a pena utilizar *Hashing* perfeito?

Análise sobre o *Hashing* perfeito

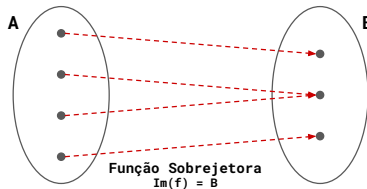
- ▶ Vale a pena utilizar *Hashing* perfeito?
 - ▶ A resposta é **depende**.

Análise sobre o *Hashing* perfeito

- ▶ Vale a pena utilizar *Hashing* perfeito?
 - ▶ A resposta é **depende**.
- ▶ Mas há outra estratégia?

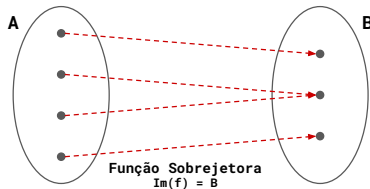
Análise sobre o *Hashing* perfeito

- ▶ Vale a pena utilizar *Hashing* perfeito?
 - ▶ A resposta é **depende**.
- ▶ Mas há outra estratégia?
 - ▶ Sim.
 - ▶ *Hashing* imperfeito. Há um problema, qual?



Análise sobre o *Hashing* perfeito

- ▶ Vale a pena utilizar *Hashing* perfeito?
 - ▶ A resposta é **depende**.
- ▶ Mas há outra estratégia?
 - ▶ Sim.
 - ▶ *Hashing* imperfeito. Há um problema, qual?
 - ▶ Diferentes chaves x_1 e x_2 pertencentes a A podem usar uma função *Hash* que resultam na **mesma saída**.

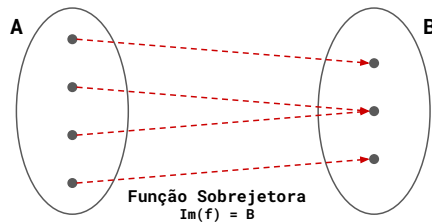


Hashing Imperfeito

Hashing Imperfeito

Considerando:

- ▶ Duas chaves diferentes x_1 e x_2 .
- ▶ Ambas pertencentes a um conjunto A .
- ▶ A função utilizada pode fornecer resultados **iguais**.



Hashing x Colisões

- ▶ Qualquer que seja a função de transformação, **algumas colisões irão ocorrer fatalmente**, e tais colisões têm de ser resolvidas de alguma forma.
- ▶ Mesmo que se obtenha uma função de transformação que distribua os registros de forma uniforme entre as entradas da tabela, existe uma alta probabilidade de haver colisões.

Colisões e o paradoxo do aniversário

- ▶ O **paradoxo do aniversário** (Feller, 1968, p. 33)
 - ▶ Em um grupo de 23 ou mais pessoas, existe uma chance maior do que **50%** de que 2 pessoas comemorem o aniversário no mesmo dia.
- ▶ Assim, se for utilizada uma função de transformação uniforme que enderece 23 chaves randômicas em uma tabela de tamanho 365, a probabilidade de que haja colisões é **maior do que 50%**.

Probabilidade de colisões

- ▶ A probabilidade p de se inserir N itens consecutivos sem colisão em uma tabela de tamanho M é:

$$\begin{aligned} p &= \frac{M-1}{M} \times \frac{M-2}{M} \times \dots \times \frac{M-N+1}{M} \\ &= \prod_{i=1}^N \frac{M-i+1}{M} = \frac{M!}{(M-N)!M^N} \end{aligned} \quad (1)$$

Probabilidade de não ocorrer colisões

- ▶ Alguns valores da probabilidade p de **não** ocorrer colisões para N valores, com $M = 365$.

N	p
10	0,883
22	0,524
23	0,493
30	0,303

- ▶ Para N pequeno a probabilidade p pode ser **aproximada** por $p \approx \frac{N(N-1)}{730}$.
- ▶ Por exemplo, para $N = 10$ então $p \approx 87,7\%$.

Funções de Transformação comuns

Função de Transformação mais usada

- Usa o resto da divisão por M :

$$h(K) = K \% M \quad (\text{em linguagem C})$$

onde K é um inteiro correspondente à chave e M é o tamanho da tabela.

Função de Transformação mais usada - Cuidados na escolha do valor de M

- ▶ M deve **preferencialmente** ser um número primo, mas não qualquer primo: devem ser evitados os números primos obtidos a partir de

$$b * i \pm j$$

onde b é a base do conjunto de caracteres (geralmente $b = 64$ para BCD, 128 para ASCII, 256 para EBCDIC, ou 100 para alguns códigos decimais), e i e j são pequenos inteiros.

- ▶ Exemplo: $b = 100$, $i = 1$, $j = 1$ (101)

Transformações de Chaves não numéricas

- ▶ As chaves **não numéricas** devem ser transformadas em números:

$$K = \sum_{i=1}^n Chave[i] \times p[i],$$

onde:

- ▶ n é o número de caracteres da chave.
- ▶ $Chave[i]$ corresponde à representação ASCII do i -ésimo caractere da chave.
- ▶ $p[i]$ é um inteiro de um conjunto de pesos gerados randomicamente para $1 \leq i \leq n$.

Transformações de Chaves não numéricas

- Vantagem de se usar pesos:

Dois conjuntos diferentes de pesos $p_1[i]$ e $p_2[i]$, $1 \leq i \leq n$



duas funções diferentes de transformação $h_1(K)$ e $h_2(K)$.

Implementação da função para gerar os pesos

- Função que gera um peso para cada caractere de uma chave constituída de n caracteres:

```
1 /* Gera valores randomicos entre 1 e 10.000 */
2 void geraPesos(int p[], int n) {
3     int i;
4     // utilizando o tempo como semente de numeros aleatorios
5     srand(time(NULL));
6     for (i = 0; i < n; i++) {
7         p[i] = 1 + (int) (10000.0 * rand() / RAND_MAX);
8     }
9 }
```

Implementação da função de transformação

```
1 /* Funcao de hash que retorna o indice (numero inteiro)
2  * de uma chave (string) */
3 int h(char *chave, int p[], int m, int tam_p) {
4     unsigned int soma = 0;
5     int i, stringSize = strlen(chave);
6
7     for (i = 0; i < stringSize; i++)
8         soma += (unsigned int) chave[i] * p[i % tam_p];
9
10    return (soma % m);
11 }
```

Resolvendo Colisões

Sugestões?

Resolvendo Colisões

Sugestões?

- ▶ Listas Encadeadas.
- ▶ Endereçamento Aberto.

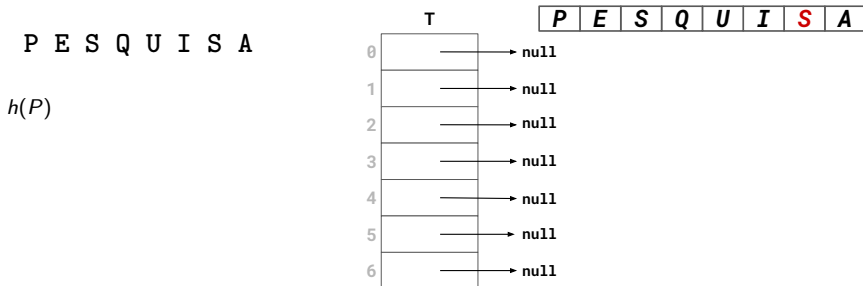
Listas Encadeadas

Listas Encadeadas

- ▶ Uma das formas de resolver as **colisões** é simplesmente construir uma lista linear encadeada para cada endereço da tabela.
- ▶ Assim, todas as chaves com mesmo endereço são encadeadas em uma lista linear.

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

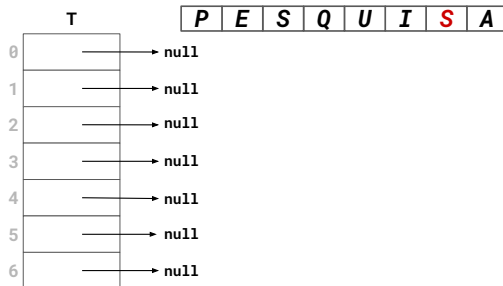


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16)$$

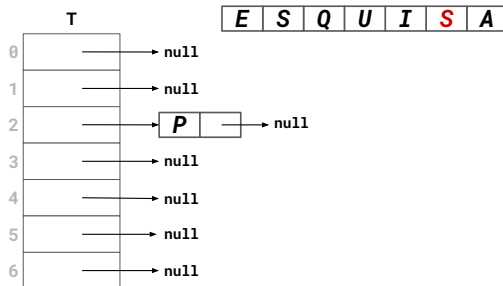


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16) = 2$$

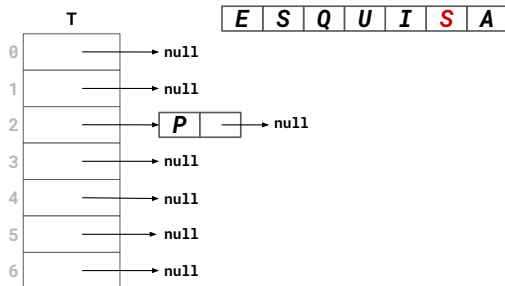


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) & \end{aligned}$$

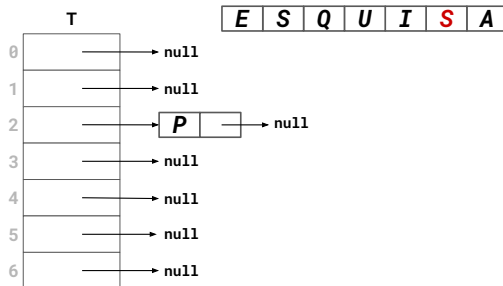


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) \end{aligned}$$



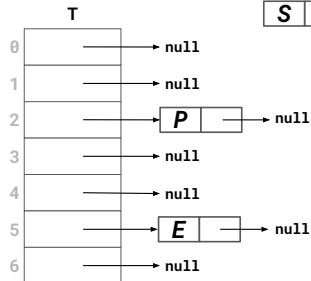
Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$



S Q U I S A

Listas Encadeadas - Exemplo

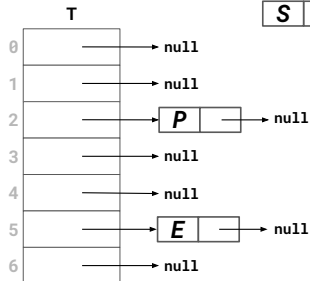
- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S)$$



S Q U I S A

Listas Encadeadas - Exemplo

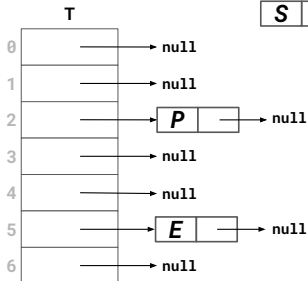
- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S) = h(19)$$



S Q U I S A

Listas Encadeadas - Exemplo

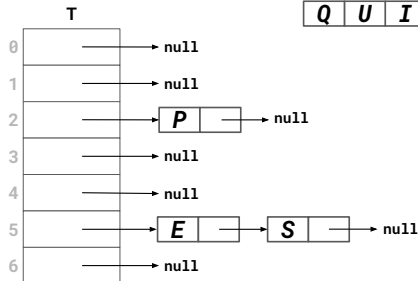
- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S) = h(19) = 5$$



Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

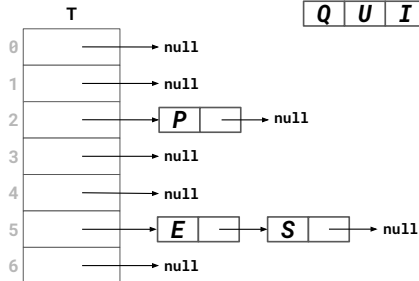
P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S) = h(19) = 5$$

$$h(Q)$$



Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

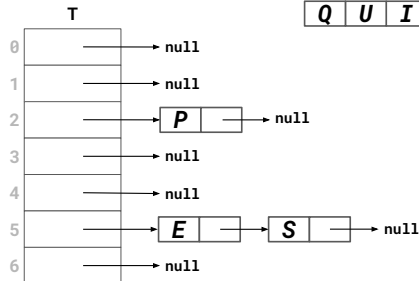
P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S) = h(19) = 5$$

$$h(Q) = h(17)$$



Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

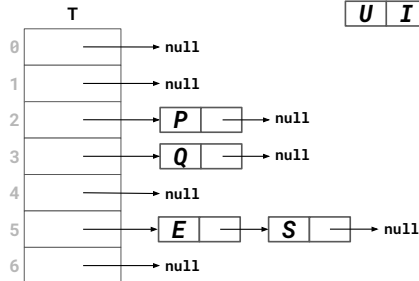
P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S) = h(19) = 5$$

$$h(Q) = h(17) = 3$$



Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

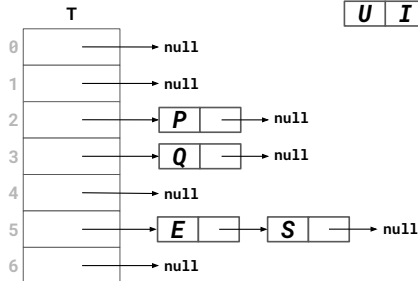
$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

$$h(S) = h(19) = 5$$

$$h(Q) = h(17) = 3$$

$$h(U)$$

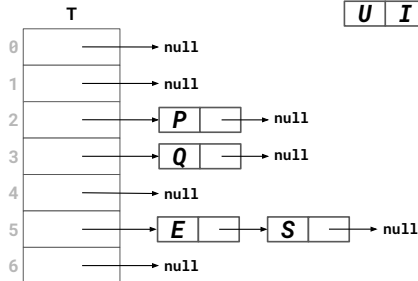


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned}h(P) &= h(16) = 2 \\h(E) &= h(5) = 5 \\h(S) &= h(19) = 5 \\h(Q) &= h(17) = 3 \\h(U) &= h(21)\end{aligned}$$

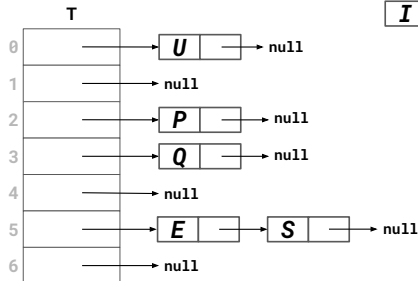


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \end{aligned}$$



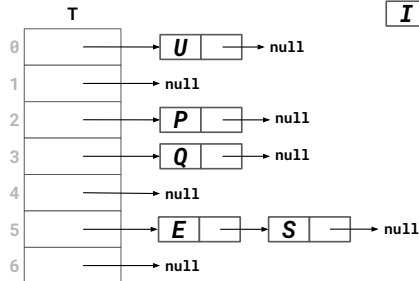
I S A

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \\ h(I) &= h(9) = 2 \end{aligned}$$



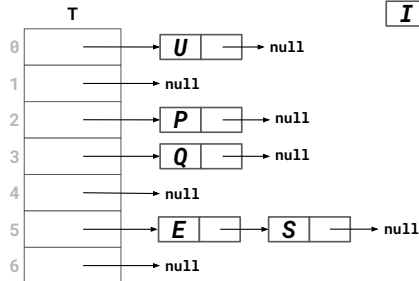
I S A

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \\ h(I) &= h(9) \end{aligned}$$



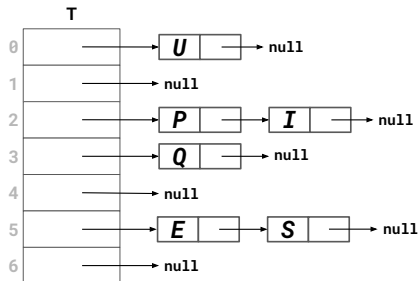
I S A

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned}h(P) &= h(16) = 2 \\h(E) &= h(5) = 5 \\h(S) &= h(19) = 5 \\h(Q) &= h(17) = 3 \\h(U) &= h(21) = 0 \\h(I) &= h(9) = 2\end{aligned}$$



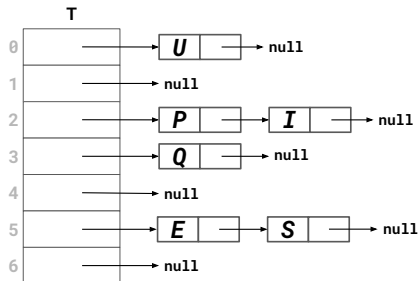
S A

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \\ h(I) &= h(9) = 2 \\ h(S) & \end{aligned}$$



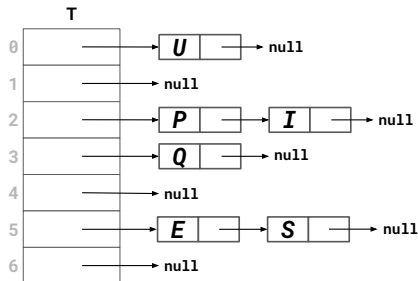
S A

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \\ h(I) &= h(9) = 2 \\ h(S) &= h(19) \end{aligned}$$



S A

Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$h(P) = h(16) = 2$$

$$h(E) = h(5) = 5$$

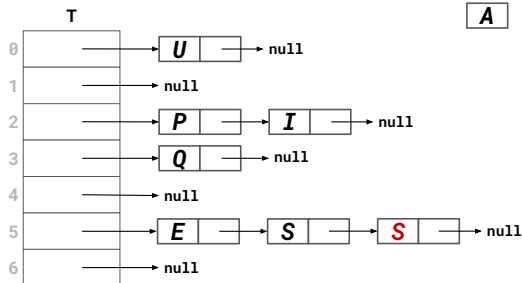
$$h(S) = h(19) = 5$$

$$h(Q) = h(17) = 3$$

$$h(U) = h(21) = 0$$

$$h(I) = h(9) = 2$$

$$h(S) = h(19) = 5$$

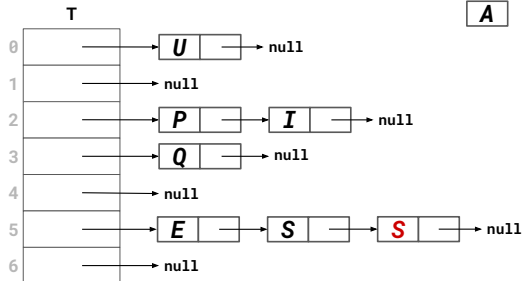


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

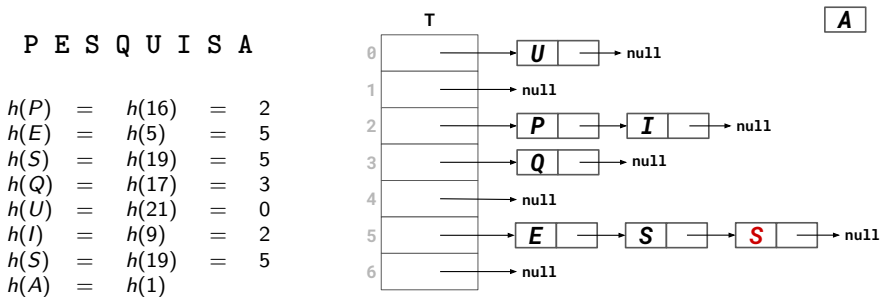
P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \\ h(I) &= h(9) = 2 \\ h(S) &= h(19) = 5 \\ h(A) & \end{aligned}$$



Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

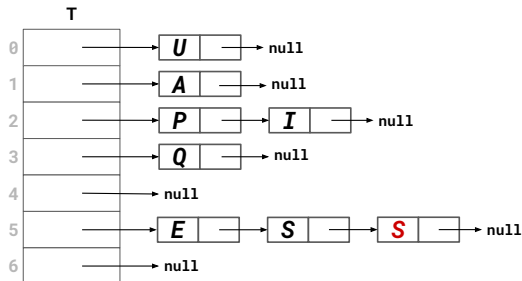


Listas Encadeadas - Exemplo

- Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação $h(chave) = chave \bmod M$ é utilizada para $M = 7$, o resultado da inserção das chaves abaixo na tabela é o seguinte:

P E S Q U I S A

$$\begin{aligned} h(P) &= h(16) = 2 \\ h(E) &= h(5) = 5 \\ h(S) &= h(19) = 5 \\ h(Q) &= h(17) = 3 \\ h(U) &= h(21) = 0 \\ h(I) &= h(9) = 2 \\ h(S) &= h(19) = 5 \\ h(A) &= h(1) = 1 \end{aligned}$$



TAD Lista Encadeada

```

1  #define N 16 // tamanho da chave (string)
2
3  typedef char Chave[N];
4
5  typedef struct {
6      /* outros componentes */
7      Chave chave;
8  } Item;
9
10 typedef struct celula {
11     struct celula *pProx;
12     Item item;
13 } Celula;
14
15 typedef struct {
16     Celula *pPrimeiro, *pUltimo;
17 } Lista;
    
```

TAD Hash

```

1  #include "lista.h"
2
3  typedef struct {
4      int n; // numero de itens na hash
5      int nro_listas; // tamanho do array de listas
6      int nro_pesos; // tamanho do array de pesos
7      int *p; // array de pesos
8      Lista *v; // array de listas
9  } Hash;
10
11 void HashInicia(Hash *hash, int nro_listas, int nro_pesos);
12 int HashH(Hash *hash, Chave chave);
13 int HashPesquisa(Hash *hash, Chave chave, Item *x);
14 Celula *HashPesquisaCelula(Hash *hash, Chave chave);
15 bool HashInsere(Hash *hash, Item x);
16 bool HashRemove(Hash *hash, Item *x);

```

Função *Hash* para Transformação de uma chave

```
1 /* Função de hash que retorna o índice (número inteiro)
2  * de uma chave (string) */
3 int HashH(Hash *hash, Chave chave) {
4     int i;
5     unsigned int soma = 0;
6     int comp = strlen(chave);
7
8     for (i = 0; i < comp; i++)
9         soma += (unsigned int) chave[i] * hash->p[i % hash->nro_pesos];
10
11     return (soma % hash->nro_listas);
12 }
```

Pesquisa em uma *Hash*

```
1 /* Retorna se a pesquisa foi bem sucedida e o item (x) por meio
2  * de passagem por referência */
3 int HashPesquisa(Hash *hash, Chave chave, Item *x) {
4     Celula *aux = HashPesquisaCelula(hash, chave);
5     if (aux == NULL) {
6         return 0;
7     }
8     *x = aux->pProx->item;
9     return 1;
10 }
```

Pesquisa na Lista Encadeada de uma Hash

```

1  /* Retorno o ponteiro apontando para a celula anterior da lista */
2  Celula *HashPesquisaCelula(Hash *hash, Chave chave) {
3      int i = HashH(hash, chave);
4      Celula *aux;
5
6      if (ListaEhVazia(&hash->v[i]))
7          return NULL; // pesquisa sem sucesso
8
9      aux = hash->v[i].pPrimeiro;
10     while (aux->pProx->pProx != NULL &&
11            strcmp(chave, aux->pProx->item.chave) != 0)
12         aux = aux->pProx;
13
14     if (!strcmp(chave, aux->pProx->item.chave, sizeof(Chave)))
15         return aux;
16     else
17         return NULL; // pesquisa sem sucesso
18 }
```

Inserção e remoção de um item em uma Hash

```

1 bool HashInsere(Hash *hash, Item x) {
2     if (HashPesquisaCelula(hash, x.chave) == NULL) {
3         ListaInsere(&hash->v[HashH(hash, x.chave)], x);
4         hash->n++;
5         return true;
6     }
7     return false;
8 }
9
10 bool HashRemove(Hash *hash, Item *x) {
11     Celula *aux = HashPesquisaCelula(hash, x->chave);
12
13     if (aux == NULL)
14         return false;
15     ListaRemove(&hash->v[HashH(hash, x->chave)], aux, x);
16     hash->n--;
17     return true;
18 }

```

Análise de *Hashs* implementadas com Listas Encadeadas

- ▶ Assumindo que qualquer item do conjunto tem **igual probabilidade de ser endereçado** para qualquer entrada de T , então o **comprimento** esperado de cada lista encadeada é N/M , onde:
 - ▶ N representa o número de registros na tabela.
 - ▶ M representa o tamanho da tabela.
- ▶ Logo: as operações **Pesquisa**, **Inserere** e **Retira** custam $O(1 + N/M)$ operações em média, onde a constante 1 representa o tempo para encontrar a entrada (lista) na tabela e N/M o tempo para percorrer a lista.
- ▶ Se os valores forem bem distribuídos (poucas colisões) e N for igual a M , teremos que:
 - ▶ Pesquisa, inserção e remoção serão $O(1)$.

Considerações Finais

Conclusão

- ▶ Implementação para *Hash* por meio de Lista encadeada.
- ▶ São armazenados em Tabelas *Hash* uma coleção de registros com chaves.
- ▶ A posição de inserção de um novo registro depende do valor resultante da função *Hash*.
- ▶ A escolha do que fazer quando há uma colisão, depende da estratégia de implementação.
- ▶ Geralmente, procurar por uma chave é rápido.

Bibliografia

Bibliografia

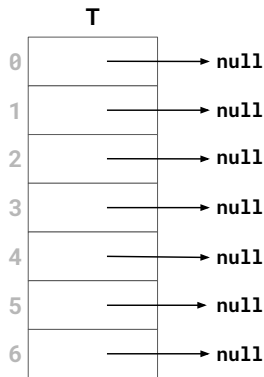
Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:

 Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.
Introduction to Algorithms.
The MIT Press, 2011.

Exercício

Mostre o passo-a-passo de como seria a inserção do seu nome onde cada letra é uma chave na tabela ao lado.

A = h(1)	B = h(2)	C = h(3)
D = h(4)	E = h(5)	F = h(6)
G = h(7)	H = h(8)	I = h(9)
J = h(10)	K = h(11)	L = h(12)
M = h(13)	N = h(14)	O = h(15)
P = h(16)	Q = h(17)	R = h(18)
S = h(19)	T = h(20)	U = h(21)
V = h(22)	W = h(23)	X = h(24)
Y = h(25)	Z = h(26)	



Considere a função *Hash* e $M = 7$: $h(chave) = chave \bmod M$